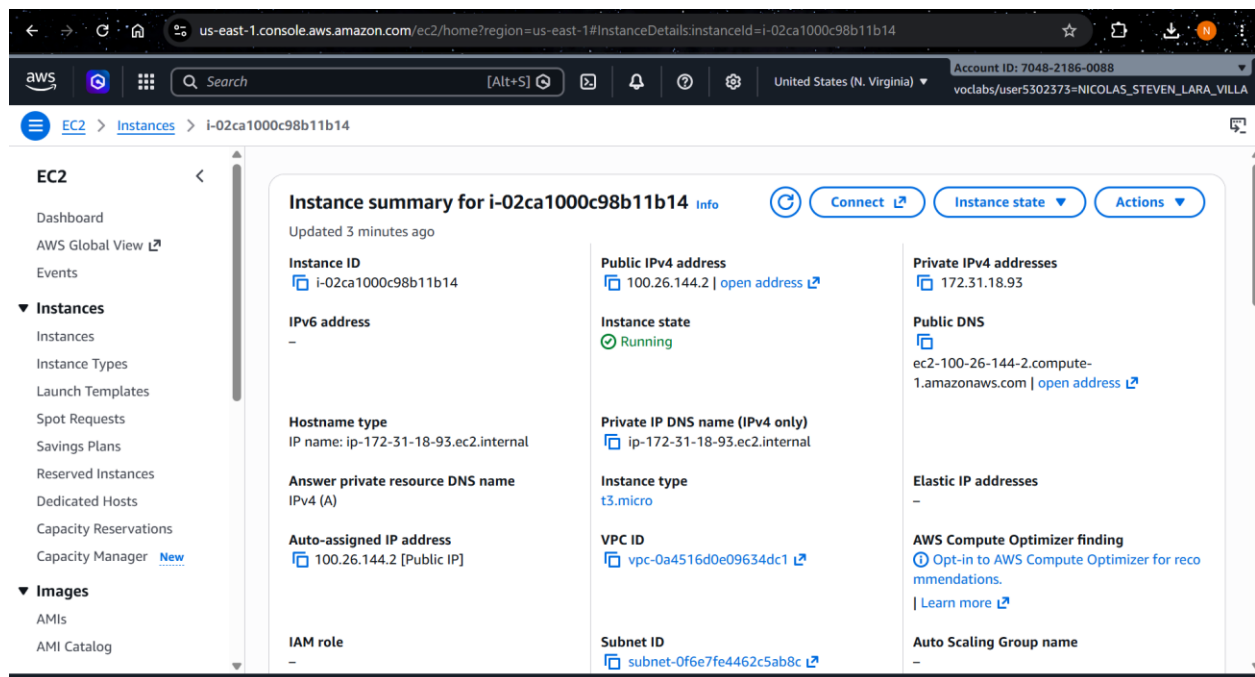


1. Pantallazo de la consola de AWS EC2 con la máquina en ejecución



2. Pantallazo y explicación de salida de Docker hello-world

```
PS C:\Users\nilara> ssh -i "C:\Users\nilara\Downloads\Taller_8.pem" -o StrictHostKey
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1017-aws x86_64)
```

```
* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro
```

System information as of Sun Aug 30 19:40:14 UTC 2026

System load:	0.16	Temperature:	-273.1 C
Usage of /:	13.5% of 18.33GB	Processes:	116
Memory usage:	31%	Users logged in:	0
Swap usage:	0%	IPv4 address for ens5:	172.31.18.93

* Ubuntu Pro delivers the most comprehensive open source security and compliance features.

<https://ubuntu.com/aws/pro>

Expanded Security Maintenance for Applications is not enabled.

107 updates can be applied immediately.

88 of these updates are standard security updates.

To see these additional updates run: `apt list --upgradable`

Enable ESM Apps to receive additional future security updates.

See <https://ubuntu.com/esm> or run: `sudo pro status`

```
ubuntu@ip-172-31-18-93:~$ sudo docker run hello-world
```

```
Unable to find image 'hello-world:latest' locally
```

```
latest: Pulling from library/hello-world
```

```
4f55086f7dd0: Pull complete
```

```
d5e71e642bf5: Download complete
```

```
Digest: sha256:5dd0d3e6e255913fc30f90b9f2b1d359cc2cbdb48090cc4b65f1676e203243cc
```

```
Status: Downloaded newer image for hello-world:latest
```

```

ubuntu@ip-172-31-18-93:~$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:5dd0d3e6e255913fc30f90b9f2b1d359cc2cbdb48090cc4b65f1676e203243cc
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

```

3. Explicación de Dockerfile

El Dockerfile define los pasos para construir la imagen del contenedor de la API:

- FROM python:3.12-slim: Define la imagen base del contenedor: Python 3.12 en su versión ligera (slim), basada en Debian.
- RUN adduser --disabled-password api-user: Crea un usuario sin contraseña llamado api-user para ejecutar la aplicación con menor privilegio que root.
- RUN apt-get install build-essential: Instala compiladores C/C++ necesarios para construir algunas dependencias de Python.
- WORKDIR /opt/bankchurn-api: Establece el directorio de trabajo dentro del contenedor donde se ejecutarán los siguientes comandos.
- ADD ./bankchurn-api /opt/bankchurn-api/: Copia el código fuente de la API desde el sistema hospedador al contenedor.
- RUN pip install -r requirements.txt: Instala todas las dependencias Python de la aplicación definidas en el archivo de requisitos.
- RUN chmod +x run.sh: Da permisos de ejecución al script de arranque.

- RUN chown -R api-user: Transfiere la propiedad de los archivos al usuario api-user.
- USER api-user: A partir de este punto el contenedor corre como api-user, no como root.
- EXPOSE 8001: Indica que el contenedor escucha en el puerto 8001.
- CMD ["bash", "./run.sh"]: Comando que se ejecuta al iniciar el contenedor , lanza el servidor uvicorn con la API.

4. Pantallazo de salida de docker images

```
ubuntu@ip-172-31-18-93:~/taller8-api$ sudo docker images
```

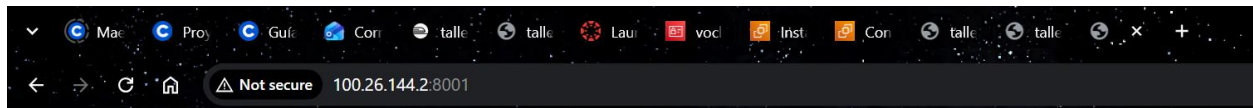
IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
bankchurn-api:latest	f8d5615f31ff	1.49GB	401MB	
hello-world:latest	5dd0d3e6e255	25.9kB	9.49kB	U

```
ubuntu@ip-172-31-18-93:~/taller8-api$
```

5. Pantallazo de salida de docker run

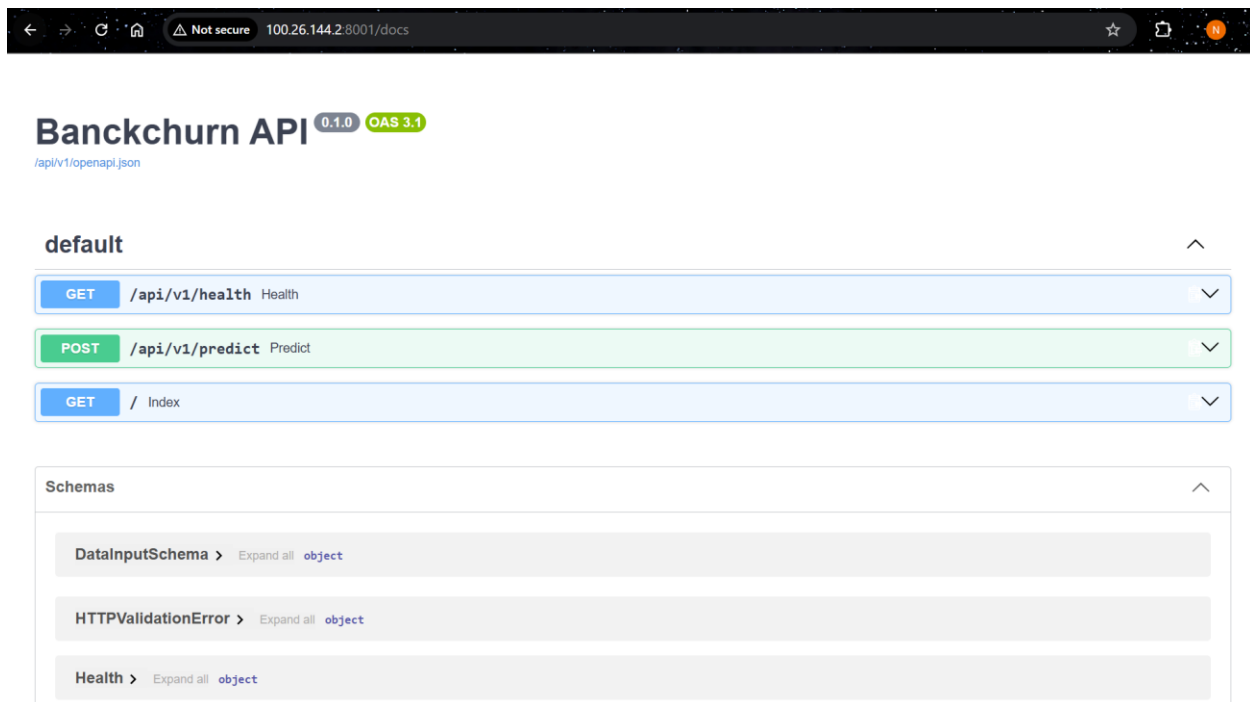
```
ubuntu@ip-172-31-18-93:~/taller8-api$ sudo docker run -p 8001:8001 -it -e PORT=8001 bankchurn-api
INFO:      Started server process [7]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8001 (Press CTRL+C to quit)
```

6. Pantallazo de navegador con API en ejecución



Welcome to the API

Check the docs: [here](#)



7. Pantallazo de salida de docker images

```
=> => naming to docker.io/library/banckchurn-dash:latest
=> => unpacking to docker.io/library/banckchurn-dash:latest
ubuntu@ip-172-31-18-93:~/taller8-dash$ sudo docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
bankchurn-api:latest	f8d5615f31ff	1.49GB	401MB	U
bankchurn-dash:latest	98d3051d44d5	856MB	211MB	
hello-world:latest	5dd0d3e6e255	25.9kB	9.49kB	U

```
ubuntu@ip-172-31-18-93:~/taller8-dash$
```

8. Pantallazo de salida de docker run

```
ubuntu@ip-172-31-18-93:~/taller8-dash$ sudo docker run -p 8050:8050 -it -e PORT=8050 -e API_URL=100.26.144.2 -e API_PORT=8001 bankchurn-dash
[2026-08-30 19:53:41 +0000] [7] [INFO] Starting gunicorn 26.2.0
[2026-08-30 19:53:41 +0000] [7] [INFO] Listening at: http://0.0.0.0:8050 (7)
[2026-08-30 19:53:41 +0000] [7] [INFO] Using worker: sync
[2026-08-30 19:53:41 +0000] [8] [INFO] Booting worker with pid: 8
[2026-08-30 19:53:42 +0000] [7] [INFO] Control socket listening at /home/dash-user/.gunicorn/gunicornctl
```

9. Pantallazo de tablero y terminales de los dos contenedores en ejecución

← → ↻ 🏠 ⚠ Not secure 100.26.144.2:8050 ☆ 📄 🔍

Ingrese la información del cliente:

Número de meses inactivo en los últimos 12 meses:

Número de servicios tomados por el cliente:

Valor total de las transacciones:

Número de transacciones:

ALTO riesgo de abandono

← → ↻ 🏠 ⚠ Not secure 100.26.144.2:8050 🔍 ☆ 📄 🔍

Ingrese la información del cliente:

Número de meses inactivo en los últimos 12 meses:

Número de servicios tomados por el cliente:

Valor total de las transacciones:

Número de transacciones:

ALTO riesgo de abandono

```
raise err
File "/usr/local/lib/python3.12/site-packages/dash/_callback.py", line 712, in a
dd_context
    output_value = _invoke_callback(func, *func_args, **func_kwargs) # type: ignore[reportArgumentType]
File "/usr/local/lib/python3.12/site-packages/dash/_callback.py", line 59, in _i
nvoke_callback
    return func(*args, **kwargs) # %% callback invoked %%
File "/opt/app/app.py", line 71, in update_output_div
    "Total_Trans_Amt": int(transac),
TypeError: int() argument must be a string, a bytes-like object or a real number,
not 'NoneType'
2026-08-30 19:56:26.041 | INFO | app:update_output_div:83 - Response: {'errors
': None, 'version': '0.0.1', 'predictions': [1.0]}
2026-08-30 19:56:26.239 | INFO | app:update_output_div:83 - Response: {'errors
': None, 'version': '0.0.1', 'predictions': [1.0]}
2026-08-30 19:56:29.663 | INFO | app:update_output_div:83 - Response: {'errors
': None, 'version': '0.0.1', 'predictions': [1.0]}
[2026-08-30 19:57:28 +0000] [7] [INFO] Handling signal: winch

ubuntu@ip-172-31-18-93: ~/t  × + ▾
uts: [DataInputSchema(Customer_Age=57, Total_Amt_Chng_Q4_Q1=0.712, Total_Relations
hip_Count=1, Total_Revolving_Bal=0.0, Total_Ct_Chng_Q4_Q1=0.843, Total_Trans_Ct=5.
0, Total_Trans_Amt=5.0, Months_Inactive_12_mon=1, Contacts_Count_12_mon=2)]
2026-08-30 19:56:26.036 | INFO | app.api:predict:45 - Prediction results: [np.
int64(1)]
2026-08-30 19:56:26.039 | INFO | logging:callHandlers:1762 - 100.26.144.2:5990
0 - "POST /api/v1/predict HTTP/1.1" 200
2026-08-30 19:56:28.157 | INFO | app.api:predict:38 - Making prediction on inp
uts: [DataInputSchema(Customer_Age=57, Total_Amt_Chng_Q4_Q1=0.712, Total_Relations
hip_Count=3, Total_Revolving_Bal=0.0, Total_Ct_Chng_Q4_Q1=0.843, Total_Trans_Ct=5.
0, Total_Trans_Amt=5.0, Months_Inactive_12_mon=1, Contacts_Count_12_mon=2)]
2026-08-30 19:56:28.229 | INFO | app.api:predict:45 - Prediction results: [np.
int64(1)]
2026-08-30 19:56:28.229 | INFO | logging:callHandlers:1762 - 100.26.144.2:5991
6 - "POST /api/v1/predict HTTP/1.1" 200
2026-08-30 19:56:29.593 | INFO | app.api:predict:38 - Making prediction on inp
uts: [DataInputSchema(Customer_Age=57, Total_Amt_Chng_Q4_Q1=0.712, Total_Relations
hip_Count=3, Total_Revolving_Bal=0.0, Total_Ct_Chng_Q4_Q1=0.843, Total_Trans_Ct=5.
0, Total_Trans_Amt=5.0, Months_Inactive_12_mon=1, Contacts_Count_12_mon=2)]
2026-08-30 19:56:29.663 | INFO | app.api:predict:45 - Prediction results: [np.
int64(1)]
2026-08-30 19:56:29.664 | INFO | logging:callHandlers:1762 - 100.26.144.2:5993
0 - "POST /api/v1/predict HTTP/1.1" 200
```

10. Comparación archivos Dockerfile

Comparación Dockerfiles , numeral 2.11 (3 diferencias sustanciales):

1. Puerto expuesto:

La API expone el puerto 8001 (EXPOSE 8001), mientras que el tablero expone el puerto 8050 (EXPOSE 8050). Esto refleja que cada servicio escucha en un puerto distinto según su función: la API recibe solicitudes de predicción y el tablero sirve la interfaz web.

2. Directorio de trabajo y código copiado:

La API define WORKDIR /opt/bankchurn-api y copia ./bankchurn-api, mientras que el tablero usa WORKDIR /opt/app y copia ./app. Cada contenedor tiene una estructura interna independiente ajustada al proyecto que aloja.

3. Usuario de ejecución y dependencias instaladas:

La API crea y corre como api-user, e instala dependencias que incluyen el modelo de ML (model_abandono-0.0.1-py3-none-any.whl) y el servidor uvicorn (ASGI). El tablero crea dash-user e instala dash, gunicorn (WSGI) y requests no contiene modelo, sino que consume la API remotamente vía HTTP.